Vishal Pandey | Applied ML Research

Mixture of Experts (MoE)

19 Aug, 2025

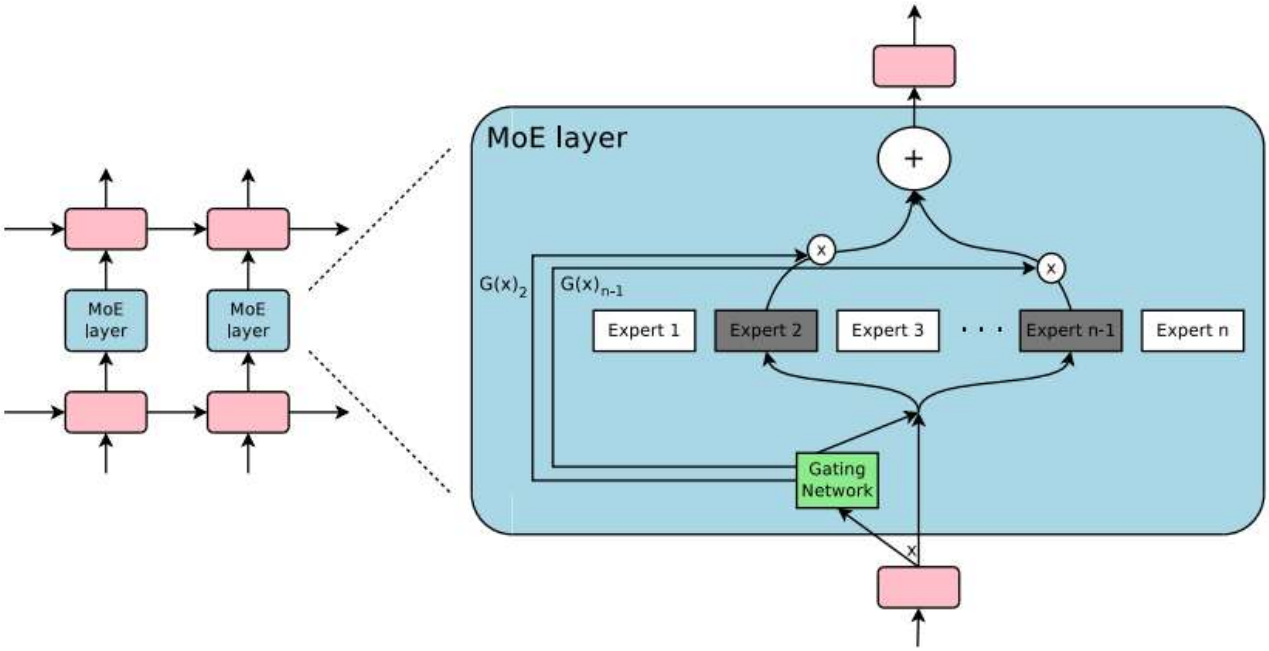
In this, we will learn about the MoEs and Sparse MoEs; both are mostly the same, but different in use cases. This can be used for the **last-minute revision for the interviews** OR **learn with maths intuition**...

This is not a new concept made by DeepSeek. The MoE was first coined by Prof. Geoffrey Hinton and later on used by DeepSeek tactically in the FFNs.

Paper Link: DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

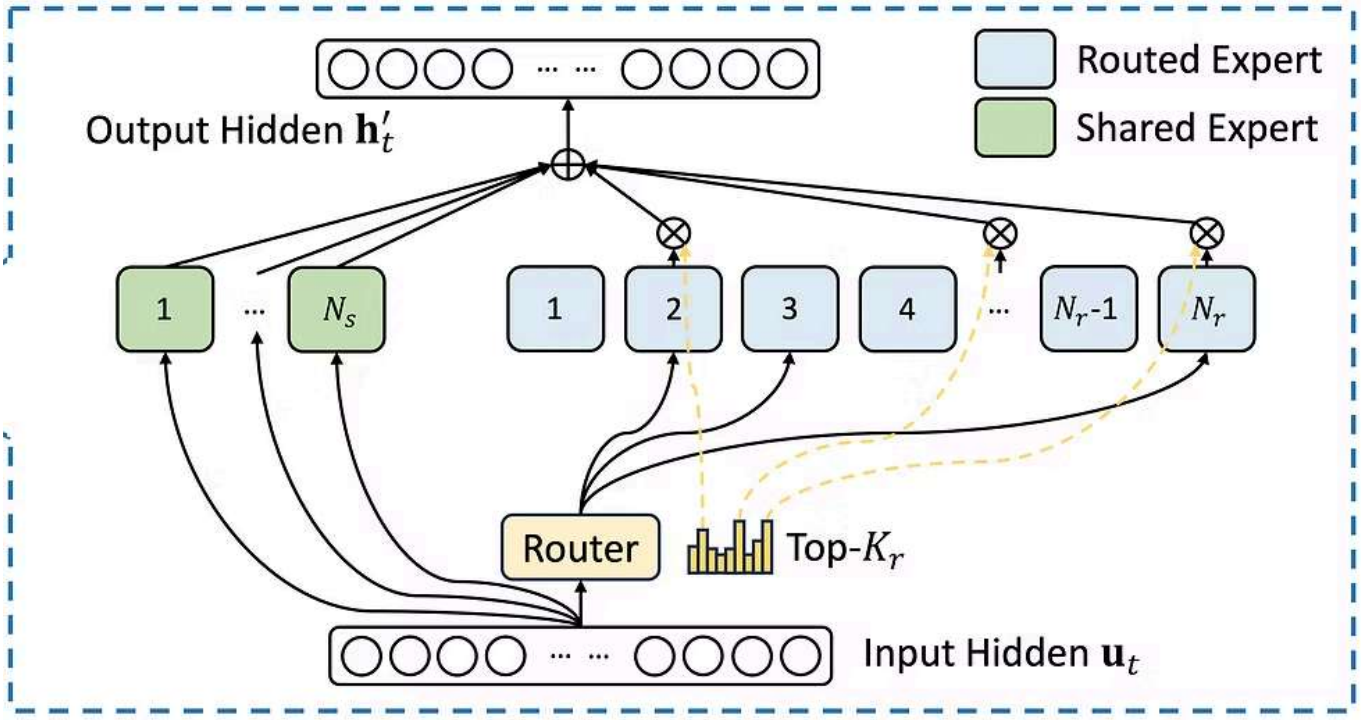
Mixture of Experts Architecture

MoEs in the Large Neural Networks

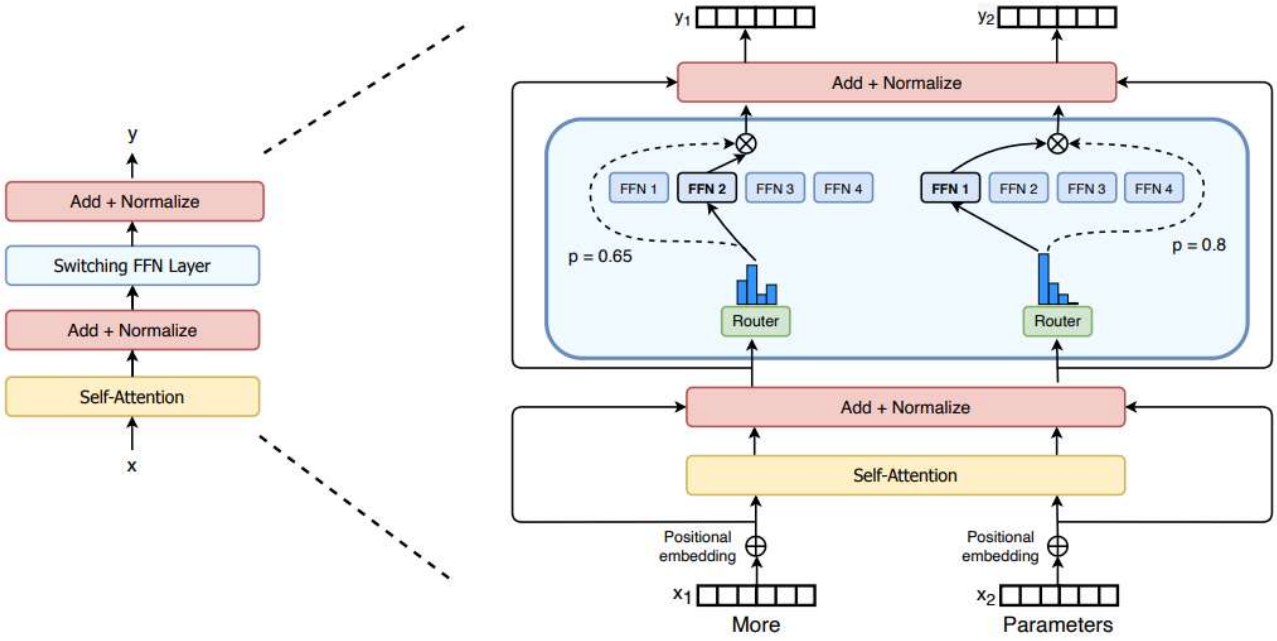


MoEs in the DeepSeek

DeepSeekMoE



MoEs in Switch Transformer



The Mixture of Experts (MoE) is a neural network architecture that uses a set of expert networks to solve tasks, while a gating network determines which experts to use for each input. Instead of using all experts for every input, the MoE model uses a sparse of experts(usually a small number) to make predictions, improving efficiency and scalability.

It is divided into three parts: **Expert Networks**, **Gating Networks**, **Combination of Outputs**

Important Formulas

- **Input**: $x \in \mathbb{R}^d$
- **Expert Network Output**: $y_i = f_i(x)$
- **Gating Network (score for expert i)**:
  $g_i(x) = \sigma(w_i^\top x)$, where σ is the sigmoid function
- **Top-K Selection (by gate scores)**:
  $\{g_{i_1}(x), g_{i_2}(x), \ldots, g_{i_K}(x)\}$, where $g_{i_j}(x)$ are the top-K values
- **Final MoE Output (Weighted Sum of Top-K Experts)**:
  $$ y_{\text{final}} = \sum_{k=1}^{K} g_{i_k}(x) \cdot f_{i_k}(x) $$

---

1. The Core Intuition of MoE

At the simplest level:

Standard neural nets: every parameter is active for every input.

MoE: only a small subset of parameters (experts) are activated depending on the input.

👉 Think of it as a committee of specialists. A question comes in → the "gating mechanism" picks which specialists should answer → the final output is the weighted combination of their answers.

This gives:

- **Scalability** → you can have billions of parameters but use only a fraction per input.
- **Specialization** → different experts handle different parts of the data distribution.

---

2. Basic Math of MoE

Let's formalize.

Suppose:

Input token representation: $x \in \mathbb{R}^d$

We have E experts, each is a function $f_i(x)$. For simplicity, think of each expert as an MLP layer.

The MoE layer output is:

$$y = \sum_{i=1}^{E} g_i(x) \cdot f_i(x)$$

Where:

- $f_i(x)$ = the output of the i-th expert
- $g_i(x)$ = gating function weight for expert i (probability-like, usually from a softmax)

Gating Function:

$$g(x) = \text{Softmax}(W_g x)$$

Here W_g is a learnable matrix.

In **sparse MoE** (most popular in practice, e.g., Switch Transformer, GLaM):

We don't use all experts.

Instead, we select the Top-K experts (say, 1 or 2) with highest gate score.

So effectively:

$$y = \sum_{i \in \text{TopK}(g(x))} g_i(x) \cdot f_i(x)$$

---

3. Intuition of Mathematics

The gating mechanism is like a router that decides which expert to call.

Training encourages experts to specialize in handling different types of tokens (e.g., numbers vs. code vs. natural language).

By using Top-K, we make the computation cheap: we don't run all experts, just a few.

Efficiency gain:

Suppose $E = 64$ experts, each with 50M parameters $\rightarrow$ 3.2B parameters total.

If we use Top-2 experts per input, the effective compute per token is ~100M parameters (instead of 3.2B).

So you get the capacity of a giant model with the compute of a smaller one.

---

4. Architectures of MoE

(A) Shazeer et al. (2017) – Sparsely Gated MoE

- First popularized MoE for language models.
- Each token routed to Top-K experts.
- Introduced load balancing loss to prevent some experts from being overused.

(B) Switch Transformer (Google, 2021)

- **Simplification**: route each token to only one expert (Top-1).
- This reduces communication overhead and makes scaling easier.
- **Huge scaling**: trained a 1.6 trillion parameter model with MoE.

Equation:

$$y = f_{i^*}(x) \quad \text{where } i^* = \arg\max g(x)$$

---

5. Training Challenges & Solutions

Load Imbalance:

Some experts get all tokens, others idle.

Solution: Add auxiliary loss to encourage balanced usage.

Example:

$$L_{\text{balance}} = \alpha \cdot \text{Var}(\text{expert usage})$$

Routing Instability:

Gate may change suddenly → unstable learning.

Solution: Use noise, temperature scaling, or smooth routing.

Communication Overhead:

Sending tokens to multiple experts across GPUs is expensive.

Solution: Use all-to-all optimized kernels (e.g., DeepSpeed-MoE).

---

6. Backpropagation Derivation (Single Token, Single MoE Layer)

Setup & Notation

- **Input**: $x \in \mathbb{R}^d$
- **Experts**: E experts, indexed by $i = 1, \ldots, E$
- Each expert is a **2-layer FFN**:
  - $a_i = W_{1,i}x + b_{1,i}$
  - $s_i = \sigma(a_i)$
  - $h_i = W_{2,i}s_i + b_{2,i} = E_i(x)$
- **Router logits**: $\ell = W_r x + b_r \in \mathbb{R}^E$
- **Temperature**: $\tau > 0$
- **Softmax gate probabilities**: $p = \mathrm{softmax}(\ell / \tau)$
  $p_i = \frac{e^{\ell_i / \tau}}{\sum_j e^{\ell_j / \tau}}$
- **Top-K (hard mask)**: $S = \mathrm{TopK}(p)$
- **Renormalized combine weights**:
  $\alpha_i = \frac{p_i}{\sum_{j \in S} p_j} \cdot 1[i \in S]$
- **Layer output**:
  $y = \sum_{i \in S} \alpha_i h_i \in \mathbb{R}^d$
- **Loss**: $L = L(y)$
  Upstream gradient: $g = \frac{\partial L}{\partial y} \in \mathbb{R}^d$

Step 1: Gradients w.r.t. Expert Outputs and Combine Weights

Since $y = \sum_{i \in S} \alpha_i h_i$:

- For h_i (if $i \in S$): $$ \frac{\partial L}{\partial h_i} = \alpha_i g $$
- For α_i (if $i \in S$): $$ \frac{\partial L}{\partial \alpha_i} = g^\top h_i $$

Step 2: Gradients Inside Each Expert

Let $u_i = \frac{\partial L}{\partial h_i} = \alpha_i g$

Then:

- $\frac{\partial L}{\partial W_{2,i}} = u_i s_i^\top$
- $\frac{\partial L}{\partial b_{2,i}} = u_i$
- $\frac{\partial L}{\partial s_i} = W_{2,i}^\top u_i$
- $\frac{\partial L}{\partial a_i} = \left(\frac{\partial L}{\partial s_i}\right) \odot \sigma'(a_i)$
- $\frac{\partial L}{\partial W_{1,i}} = \left(\frac{\partial L}{\partial a_i}\right) x^\top$
- $\frac{\partial L}{\partial b_{1,i}} = \frac{\partial L}{\partial a_i}$

Expert contribution to input gradient:

$$\left.\frac{\partial L}{\partial x}\right|_{\text{expert } i} = W_{1,i}^\top \Big(\big(W_{2,i}^\top (\alpha_i g)\big) \odot \sigma'(a_i) \Big)$$

Only experts $i \in S$ receive/use these gradients.

Step 3: Gradients w.r.t. Masked Softmax Weights α

We treat α as a softmax over masked logits:

$$z_i = \begin{cases} \ell_i / \tau & i \in S \\ -\infty & i \notin S \end{cases}, \quad \alpha_i = \frac{e^{z_i}}{\sum_{j \in S} e^{z_j}} \quad \text{for } i \in S$$

Softmax Jacobian (restricted to S):

$$\frac{\partial \alpha_i}{\partial z_j} = \alpha_i(\delta_{ij} - \alpha_j), \quad i, j \in S$$

Chain rule to logits:

$$\frac{\partial \alpha_i}{\partial \ell_j} = \frac{1}{\tau}\alpha_i(\delta_{ij} - \alpha_j), \quad j \in S$$

Then:

$$\begin{aligned}
&\frac{\partial L}{\partial \ell_j} = \sum_{i \in S} \frac{\partial L}{\partial \alpha_i}\frac{\partial \alpha_i}{\partial \ell_j} \\
&= \frac{1}{\tau}\sum_{i \in S} (g^\top h_i)\alpha_i(\delta_{ij} - \alpha_j) \\
&= \frac{1}{\tau}\Big((g^\top h_j)\alpha_j - \alpha_j \sum_{i \in S} \alpha_i (g^\top h_i) \Big) \\
&= \frac{1}{\tau}\alpha_j \Big(g^\top h_j - g^\top y \Big) \\
&= \frac{1}{\tau}\alpha_j \, g^\top (h_j - y), \quad j \in S
\end{aligned}$$

And $\frac{\partial L}{\partial \ell_j} = 0$ for $j \notin S$.

Why Use δ_{ij} Instead of Just 1?

Softmax is **multi-dimensional**, not independent.

Each output depends on all logits, so the derivative is a **Jacobian**.

- If $i = j$ (self-derivative):
  $\frac{\partial \alpha_i}{\partial z_i} = \alpha_i(1 - \alpha_i)$
- If $i \neq j$ (cross-derivative):
  $\frac{\partial \alpha_i}{\partial z_j} = -\alpha_i \alpha_j$

Using δ_{ij} gives both in one clean formula:

$$\frac{\partial \alpha_i}{\partial z_j} = \alpha_i(\delta_{ij} - \alpha_j)$$

Step 4: Router Gradients

Given $\ell = W_r x + b_r$:

- $\frac{\partial L}{\partial W_r} = \left(\frac{\partial L}{\partial \ell}\right) x^\top$
- $\frac{\partial L}{\partial b_r} = \frac{\partial L}{\partial \ell}$
- Router's contribution to input gradient:

  $\left.\frac{\partial L}{\partial x}\right|_{\text{router}} = W_r^\top \left(\frac{\partial L}{\partial \ell}\right)$

Step 5: Total Input Gradient

Sum of expert and router contributions:

$$\frac{\partial L}{\partial x} = \sum_{i \in S} W_{1,i}^\top \Big(\big(W_{2,i}^\top (\alpha_i g)\big) \odot \sigma'(a_i) \Big) + W_r^\top \left(\frac{\partial L}{\partial \ell}\right)$$

---

7. What Interviewers Love to Probe in MoE Architectures

Who gets the gradient?

- **Only the selected experts** (for that token) receive gradients from the main loss.
- The **router** gets gradients **only through the selected α values** — i.e., for experts in the Top-K set.

- Unless you use **Top-1 without scaling** ("Top-1-no-scale"), the router gradient is sparse.
- **Auxiliary losses** (like load balancing or entropy loss) are often used to improve the router training signal.

Router–Input Coupling

- The input gradient includes a **router path**:

$$\frac{\partial L}{\partial x}\Big|_{\text{router}} = W_r^\top \left(\frac{\partial L}{\partial \ell} \right)$$

- This means the **routing decisions** affect not only the expert weights but also the upstream features and gradients — i.e., the router is **coupled** with input representations.

- It's not just about deciding *which expert* gets used — it also **shapes the input representation** during training.

Top-1 vs Top-K Trade-Off

- **Top-1 Routing (Switch Transformers):**
  - Only one expert per token.
  - Simpler and faster.
  - But: **No gradient signal to the router from the main loss** unless:
    - You **scale the expert output** by the probability p_i.
    - Or add a **strong auxiliary loss** to guide the router.

- **Top-K Routing:**
  - Allows richer combinations and more robust gradients.
  - Slightly more computationally expensive than Top-1.

2